

Quantum-Safe TLS 1.3 Handshake Middleware

Technical Analysis — Architecture, Security Posture, and Proxy Readiness

Project	TLS_1.3_Handshake (hybrid ML-KEM-768 / ML-DSA-65 over TLS 1.3)
Prepared for	navneetp22875@gmail.com
Basis	Direct source review: common/, client/, server/, middleware/, tests/, plus PROXY_CONVERSION_PLAN.md and PRODUCTION_READINESS_REVIEW.md
Date	2026-09-19

Bottom line: strong post-quantum cryptographic primitives (real ML-KEM-768 / ML-DSA-65 via liboqs), undermined by a default-on MITM gap in how the hybrid layer is wired together, and not yet a proxy in the traffic-forwarding sense.

1. What Is Actually Happening in This Implementation

This is a point-to-point secure channel built on a **hybrid classical + post-quantum handshake**, layered inside a standard TLS 1.3 connection — not (yet) a traffic-forwarding proxy. The flow is:

- A standard TLS 1.3 socket is opened first (`common/tls.py`), pinned to exactly TLS 1.3 (min = max = TLSv1_3).
- Inside that tunnel, a second, custom handshake runs: **ML-KEM-768** (`common/pq_kem.py`) for key exchange and **ML-DSA-65** (`common/pq_dsa.py`) for authentication — both via real `liboqs` bindings, not mocked. (A separate mock crypto module, `common/pq.py`, exists but is never imported.)
- `common/transcript.py` hashes every handshake message (role + type + length + payload, domain-separated SHA-256) into a running transcript digest, which is signed and verified by both sides.
- `common/hybrid.py`'s `HybridKeyDerivation.derive()` combines the ML-KEM shared secret, a 'TLS contribution', the transcript digest, and both peers' nonces through HKDF-SHA256 into one 32-byte master key.
- That key drives a `SecureSession` (`common/session.py`) using directional AES-256-GCM with per-direction sequence numbers folded into the AEAD associated data, for replay/reorder protection.
- Today, the only behavior on top of an established session is the server echoing `f"ACK: {message}"` back to whatever the client typed (`client/client.py` is an interactive stdin shell).

2. Why a Proxy Is the Right End Product — But Isn't Built Yet

A raw PQC handshake library has no standalone product value — it needs something to protect. The natural end product is a **gateway that terminates the hybrid handshake with an external client, then forwards plaintext to an ordinary backend** (HTTP service, database, etc.) over plain or standard TLS. This is the same shape as a TLS-terminating load balancer, with the client-facing leg upgraded to be quantum-resistant — *terminate-and-reissue*, not blind byte pass-through, because no ordinary backend speaks this bespoke protocol.

Per `PROXY_CONVERSION_PLAN.md`, none of the forwarding machinery exists today. A repo-wide search for `backend|upstream|forward|proxy_to` returns zero matches.

Missing pieces, concretely:

- Backend/upstream configuration (host, port, or multiple targets)
- An outbound dialer to a backend (plain TCP or standard TLS)
- A 'gateway' role that is simultaneously a PQ-server toward the client and a plain/TLS client toward the backend
- A bidirectional relay/pump between the two connection legs (existing loops are one-directional read-then-respond, not a two-socket pump)
- Coordinated connection teardown (closing one leg should close the other)
- Tests covering relay behavior

What already exists and is reusable as-is: the client-side handshake pipeline (`secure_connect()`, `common/middleware.py`) and the server-side mirror (`run_secure_server()`). The plan document lays out 9 dependency-ordered steps — backend config → dialer → relay primitive → gateway role → timeouts

→ CLI mode → tests — estimated at roughly 3–5 engineer-days for a minimum viable proxy, plus 2–3 more days for integration/load tests.

3. How This Implementation Differs From Other TLS 1.3 / PQC Handshakes

- **Custom handshake layered on top of TLS, not inside it.** Rather than using a TLS library's native PQC key-exchange group (e.g. OpenSSL's X25519MLKEM768 as a TLS extension), this project runs a second, independent, hand-rolled handshake — its own message types, transcript hashing, and signature scheme — as an application-layer protocol riding inside an already-established TLS 1.3 tunnel.
- **Three-way hybrid combiner.** It attempts to bind the derived session key to three inputs at once: the ML-KEM shared secret, a value derived from the outer TLS session, and the full handshake transcript — intending that compromising any single layer alone should not compromise the result (see Section 4 for why this currently falls short in practice).
- **Explicit key-confirmation step.** Both sides must prove possession of the derived key via constant-time HMAC (`hmac.compare_digest`) before the channel is trusted, rather than trusting the handshake outcome implicitly.
- **From-scratch wire format.** A custom binary protocol and serializer (`common/protocol.py`, `common/serializer.py`) rather than reuse of an existing secure-transport abstraction (e.g. Noise) — unusual, since most PQC demos simply swap the KEM inside an existing TLS/Noise library rather than building a parallel state machine end-to-end.

4. How Safe Is This Handshake, Honestly

The individual cryptographic primitives are solid, but the way they're composed today has a **critical, default-on man-in-the-middle gap**:

- `secure_connect()` defaults to `verify_server=False`, disabling outer TLS certificate/hostname verification by default (`common/middleware.py:470`).
- The 'TLS contribution' fed into the hybrid key derivation is **not** a real RFC 5705 TLS exporter secret — the code's own docstring in `common/tls_session.py` says so. It is a SHA-256 hash of *public*, easily-guessable metadata (negotiated TLS version, cipher name, session-reused flag), contributing essentially zero secret entropy or authentication to the hybrid key.
- Combined effect: with outer TLS verification off by default and the 'hybrid' contribution carrying no real secrecy, the ML-KEM/ML-DSA public keys are trust-on-first-use with no external root of trust (no CA pinning). An active attacker can transparently substitute their own keys on both legs and complete a handshake both sides believe is authenticated — no downgrade trick required, just the shipped defaults.
- Separately, **test coverage is zero**: all seven files under `tests/` are 0 bytes, named for exactly the security-critical paths (handshake, replay, tampering, signature, transcript, KEM). Nothing currently verifies these guarantees hold under adversarial input.

What is genuinely solid, and should not regress: correct, spec-conformant use of real liboqs bindings for ML-KEM-768 and ML-DSA-65 (including checking signature-verification return values before trusting them); CSPRNG-based nonces (`os.urandom`, not Python's `random`); strong transcript binding with domain separation; constant-time HMAC key confirmation; and robust replay/tampering protection via per-direction

monotonic sequence numbers folded into AEAD associated data, with fail-closed error handling throughout the session layer.

Verdict: **not safe to deploy with current defaults**. Fixing `verify_server=True` as the default and either implementing a real TLS exporter or dropping the 'hybrid' claim in favor of PQ-auth-plus-real-PKI would close the critical gap; the surrounding design is otherwise strong.

5. What Real-World Problem This Is Meant to Solve

The motivating threat is "**harvest-now, decrypt-later**": an adversary recording today's classically-encrypted TLS traffic in order to decrypt it retroactively once a cryptographically relevant quantum computer exists, at which point Shor's algorithm would break the RSA/ECDHE key exchanges that classical TLS relies on today.

By running ML-KEM-768 (a NIST-standardized, quantum-resistant key-encapsulation mechanism) and ML-DSA-65 (a quantum-resistant signature scheme) alongside classical TLS 1.3, the intent is that traffic captured now stays protected even after quantum computers arrive — provided the classical leg was also secure at the time of capture, which is the whole point of a hybrid design (either layer alone should suffice for forward secrecy).

In practice, two gaps stand between this codebase and actually solving that problem in a real deployment:

- **No proxy/backend-forwarding capability yet** (Section 2) — it cannot sit in front of a real service today, only echo test strings between two peers that both run this exact code.
- **The MITM defaults and fake TLS-exporter** (Section 4) — right now the classical-auth layer undermines the quantum-safe key exchange regardless of how strong ML-KEM/ML-DSA themselves are; an attacker doesn't need to break post-quantum crypto to defeat this system as currently configured, they just need to be on-path during the handshake.

Sources: direct code review of `common/handshake.py`, `common/hybrid.py`, `common/pq_kem.py`, `common/pq_dsa.py`, `common/tls_session.py`, `common/middleware.py`, `common/session.py`, `common/transcript.py`, `client/client.py`, `server/server.py`, `middleware/main.py`; and the repository's own `PROXY_CONVERSION_PLAN.md` and `PRODUCTION_READINESS_REVIEW.md`.